

Debriefing : client, serveur, proxy, pair

Client-serveur

Un *serveur* est un processus qui passe son temps à attendre qu'un processus *client* lui demande un service, et lorsqu'un client lui demande un service, lui rend ce service.

Imaginez-vous dans un bar. Vous êtes le client, la personne derrière le bar est le serveur. Vous vous approchez du comptoir, vous demandez une boisson, le serveur prépare la boisson et vous la donne.

- exemples de client HTTP (web) : **firefox**, **chromium**, **elinks**, **wget**, **curl**.
- exemples de serveurs HTTP (web) : **nginx**, **apache**.
- exemple de client SSH : **openssh-client**, lorsque vous lancez la commande **ssh** depuis votre système local.
- exemple de serveur SSH : **openssh-server**, le processus **sshd** qui tourne sur votre conteneur distant pour vous permettre de vous y connecter avec votre client SSH.

Remarque : plusieurs serveurs peuvent tourner sur un même système, ils écoutent simplement sur des sockets ou des ports différents.

Exercice : sur votre conteneur, pour savoir quels serveurs écoutent (option **-l**, **--listening**) sur quels ports au dessus du protocole TCP (option **-t**, **--tcp**), exécutez la commande :

```
$ ss -t -l
```

Puis rajoutez l'option **-n** pour voir comment le fichier **/etc/service** permet de faire le lien entre un numéro de port et le nom du service correspondant.

Proxy

Un *proxy* (fr: *mandataire*) est un processus qui sert de relais au client pour parler au serveur.

Imaginez que vous êtes assis-e à une table avec des ami-es. Vous voulez une boisson, un-e de vos ami-es se lève et va demander une boisson à votre place, la demande au serveur qui la prépare, et votre ami-e vous la rapporte. Votre ami-e joue le rôle de proxy.

- exemple : lorsque vous ajoutez l'option **DynamicForward** à votre client SSH, vous le transformez en proxy **SOCKS**, ce proxy peut être utilisé par votre client web (**firefox** ou **chromium**) pour joindre le serveur web (**nginx**) de votre conteneur.

Reverse-proxy

Un *reverse-proxy*, c'est comme un proxy, mais côté serveur.

Dans l'image du bar (chic), ça serait un commis de salle, qui récupère la commande auprès du client, la fait passer au serveur (qui prépare la boisson), et apporte la boisson au client.

Dans ce cours, nous utilisons **nginx** comme un serveur web. Ce logiciel est souvent utilisé comme reverse-proxy, de par sa capacité à encaisser de nombreuses connexions. Par exemple, plusieurs serveurs complexes peuvent tourner sur un système (**apache2**, **nodejs**, etc), nginx écoute sur les ports 80 et 443, sert les fichiers statiques (images, css, ...) directement, et redirige les requêtes des clients aux différents serveurs web selon la page demandée. Il peut aussi servir pour faire du load-balancing afin de répartir la charge entre plusieurs serveurs web qui tournent derrière.

Pair

Les communications entre processus ne sont pas forcément asymétriques comme l'est la relation client/serveur. Les réseaux symétriques sont appelés réseaux *pair à pair* (en: *peer to peer*, abrégé en **P2P**), les processus qui y participent sont appelés *pairs*.

Imaginez qu'au lieu de vous trouver dans un bar, vous vous trouvez dans un pique-nique : chaque personne apporte quelque chose à boire et le partage avec ses voisin-es.